Guion de trabajo tutorizado

Bases de datos orientadas a grafos

Gestión y visualización de datos masivos

Profesor Francisco A. Gómez Vela
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «Bases de datos Orientadas a grafos-Actividad.docx.pdf» (Campus Virtual UPO).

Objetivos

- Conocer las diferencias de rendimiento de las principales bases de datos orientadas a grafos.
- Aprender a instalar y realizar operaciones básicas con la base de datos Neo4j.

Ejercicio/Problemas

A1. (150 minutos) Establecer una comparativa (de no más de 300 palabras) entre las siguientes bases de datos orientadas a grafos: Neo4j, Microsoft Azure Cosmos DB, ArangoDB y OrientDB. Al final de la comparativa presente, de forma esquemática mediante una tabla, las ventajas e inconvenientes de cada base de datos.

A2. (10 minutos) Descargar e instalar la base de datos Neo4j.

Nos instalaremos la última versión de la Community Edition, que la podemos descargar de [1].

Tras su instalación habrá que documentar los pasos realizados así como las dificultades encontradas y cómo han sido resueltas.

A3. (200 minutos) Juan, María, Luisa, Alberto y Fernando son un grupo de personas que comparten afición a la lectura, con edades de 22, 27, 32, 29 y 47, respectivamente. Todos pertenencen a una red social en la que cada usuario deja una valoración de cada libro que se lee. En dicha red social, María es amiga de Alberto y Fernando. Luisa, de Alberto y Juan. Por último, Juan también es amigo de Alberto y Fernando.

Juan y María han leído el libro #1. Juan lo leyó el 16 de Febrero de 2015 y lo valoró con un 9 mientras que María lo leyó el 15 de Julio de 2015 y lo valoró con un 8. Fernando, Luisa y María coindicieron en la lectura de #2. Las fechas de lectura fueron el 8 de Abril de 2015, el 16 de Marzo de 2016 y el 15 de Septiembre de 2016 respectivamente. Fernando lo valoró con un 7 mientras que Luisa con un 9 y María con un 8.

Información de lo libros:

#1 El médico. Noah Gordon. Rocabolsillo, 2008. #2 La pirámide inmortal. Javier Sierra. Planeta, 2015.

1) Con la información anterior, modele dichos datos y cree una base de datos en Neo4j. Dibuje un diagrama que muestre, con claridad, el conjunto de nodos y relaciones así como las propiedades asociadas a dichos nodos y relaciones.

2) Genere las siguientes consultas en cypher:

1. ¿Quiénes leyeron el libro #2?
2. ¿Cuál es la valoración promedio de de #1? Ayuda: Use la función avg.

3) ¿Quién es el usuario de mayor edad? Ayuda: Use las palabras claves ORDER BY y LIMIT.

Entregue, en un documento, el modelo de datos y las consultas. El conjunto de instrucciones para generar la base de datos deberá entregarse en un script.

Fecha y Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada. .

Bibliografía

1. Página principal de Neo4j http://neo4j.com/

2. https://neo4j.com/developer/guide-neo4j-browser/#_installing_and_starting_neo4j_brows er

RESOLUCIÓN

A1. Comparativa: Neo4j, Azure Cosmos DB, ArangoDB y OrientDB

Neo4j es la base de datos de grafos nativa de referencia. Implementa el modelo de *grafo de propiedades* con adyacencia libre de índices: cada nodo guarda punteros físicos a sus relaciones, así que recorrer un salto cuesta lo mismo con independencia del tamaño del grafo. Usa Cypher, lenguaje declarativo en el que se basa el estándar ISO GQL. Es transaccional (ACID) y muy eficiente en recorridos profundos. Su edición Community se limita a un único servidor; el clúster y la escalabilidad horizontal son de la edición Enterprise o de AuraDB, su versión en la nube.

Azure Cosmos DB (API para Gremlin) es un servicio gestionado, multimodelo y distribuido globalmente de Microsoft. Expone grafos mediante Apache TinkerPop/Gremlin sobre un almacén de documentos particionado. Escala horizontalmente de forma elástica y ofrece SLA de latencia y varios niveles de consistencia. En cambio, las consultas que cruzan particiones y los recorridos de muchos saltos son más lentos y caros, porque se facturan en unidades de solicitud (RU). Sólo existe en la nube de Azure.

ArangoDB es multimodelo nativo (documentos, clave-valor y grafos) con un único lenguaje, AQL, que combina *joins*, búsquedas de texto y recorridos. Los grafos se guardan como colecciones de documentos y aristas indexadas, lo que da buen rendimiento en consultas mixtas. En recorridos muy profundos no alcanza a un motor nativo como Neo4j.

OrientDB también es multimodelo (documentos y grafo) y usa punteros físicos (RID) entre registros, lo que le da recorridos rápidos.Su comunidad y su mantenimiento han decaído desde su adquisición por SAP, y su creador impulsa ahora ArcadeDB como sucesor. Es arriesgado para proyectos nuevos.

En síntesis: Neo4j para análisis de relaciones intensivo, Cosmos DB para aplicaciones globales en Azure, ArangoDB cuando conviven documentos y grafos y OrientDB sólo en sistemas heredados.

(Texto de la comparativa: 294 palabras.)

Base de datos	Ventajas	Inconvenientes
Neo4j	Grafo nativo (adyacencia sin índices): recorridos muy rápidos · Cypher/GQL expresivo · ACID · gran comunidad, herramientas (Browser, Bloom) y librería GDS de algoritmos	Escalado horizontal y clúster sólo en Enterprise/Aura · licencia comercial para producción a gran escala · monomodelo
Azure Cosmos DB (Gremlin)	Totalmente gestionado · distribución global y escalado elástico · SLA de latencia y disponibilidad · multimodelo	Sólo en Azure (dependencia del proveedor) · coste por RU difícil de prever · recorridos profundos o entre particiones lentos y caros · Gremlin menos declarativo
ArangoDB	Multimodelo nativo con un solo lenguaje (AQL) · clúster y SmartGraphs · buen rendimiento en consultas mixtas documento+grafo	Recorridos profundos más lentos que en un grafo nativo · menor ecosistema · licencia BSL en versiones recientes
OrientDB	Multimodelo · punteros físicos (RID) con recorridos rápidos · SQL extendido y Gremlin · código	Desarrollo prácticamente estancado · comunidad reducida · problemas de estabilidad en clúster · poco

	abierto (Apache 2.0)	recomendable para proyectos nuevos

A2. Descarga e instalación de Neo4j

Se ha descargado la última **Neo4j Community Edition, versión 2026.08.1** para Linux (`neo4j-community-2026.08.1-unix.tar.gz`) desde la página de descargas de Neo4j [1] y se ha instalado siguiendo estos pasos:

- **Requisito de Java.** Las versiones calendario de Neo4j (2025.x/2026.x) exigen **Java 21**. La máquina usa Java 17 por defecto (para HBase y Solr), así que para Neo4j se apunta `JAVA_HOME` al OpenJDK 21, también instalado.
- **Instalación.** Basta con descomprimir el `.tar.gz` (`tar -xzf …`) y enlazarlo en `/opt/neo4j`. `NEO4J_HOME/bin` se añade al `PATH`.
- **Configuración.** En `conf/neo4j.conf` se deja la configuración por defecto: Bolt en el puerto 7687 y HTTP/Browser en el 7474. Para este entorno de pruebas local se desactiva la autenticación (`dbms.security.auth_enabled=false`). En un entorno real hay que mantenerla y fijar la contraseña inicial con `neo4j-admin dbms set-initial-password`.
- **Arranque.** `neo4j start` lanza el servidor en segundo plano. `neo4j version` y la salida del arranque confirman la versión y los directorios (datos, logs, plugins, import).

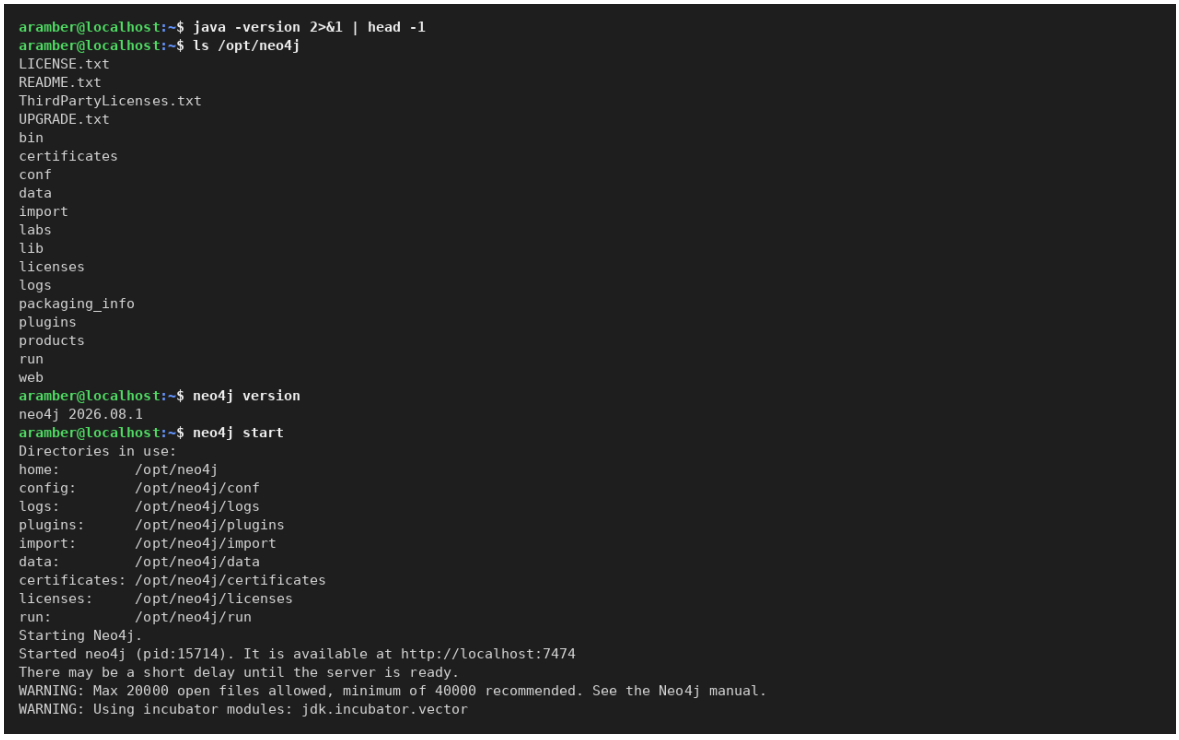


Figura 1. Versión de Java, estructura de directorios de la instalación, `neo4j version` y `neo4j start`.

Dificultades encontradas y cómo se resolvieron:

- La **versión de Java** es el requisito que más problemas suele dar: Neo4j 2026 no admite Java 17, que es el que usaban el resto de herramientas. Se resolvió fijando `JAVA_HOME` a Java 21 sólo en la sesión de Neo4j, para no afectar a las demás.

- Aviso **«Max 20000 open files allowed, minimum of 40000 recommended»**: el límite de descriptores del sistema es bajo. No impide trabajar con una base pequeña. En producción se sube con `ulimit -n 40000` o en `/etc/security/limits.conf`.
- El **Browser** web (http://localhost:7474) muestra en las versiones recientes la nueva interfaz *Query*. Al conectar hay que elegir el método de autenticación. Con la autenticación desactivada se usa «No authentication».

A3. Red social de lectores

1) Modelo de datos y diagrama

Se usa el modelo de **grafo de propiedades**:

- **Nodos `Persona`** con `nombre` y `edad`.
- **Nodos `Libro`** con `id` (#1, #2), `titulo` y `anio`. El autor y la editorial se modelan como **nodos propios** (`Autor`, `Editorial`) unidos por `ESCRITO_POR` y `PUBLICADO_POR`. Así no se repiten datos y se pueden hacer consultas como «libros de la misma editorial».
- **Relación `AMIGO_DE`** entre personas. La amistad es simétrica: se almacena **una sola vez** y se consulta sin dirección (`-[:AMIGO_DE]-`) para no duplicar información.
- **Relación `HA_LEIDO`** de Persona a Libro con las propiedades `fecha` (tipo `date`) y `valoracion`. Fecha y nota dependen del par persona-libro, así que pertenecen a la relación y no a ninguno de los nodos.
- Restricciones de unicidad sobre `Persona.nombre` y `Libro.id`, que además crean índices para localizar los nodos de inicio de las consultas.

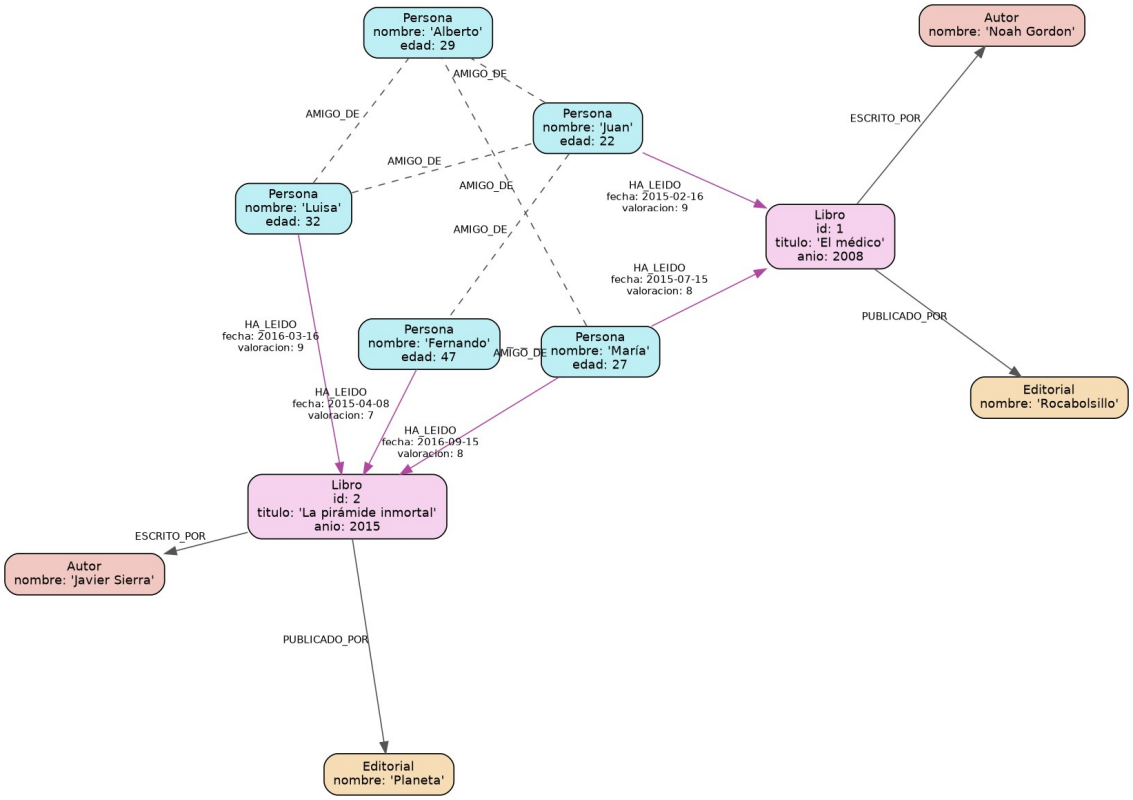


Figura 2. Diagrama del modelo con todos los nodos, relaciones y propiedades. Las líneas discontinuas son amistades (sin dirección).

Script de creación (se entrega también como fichero independiente `crear_bd_lectores.cypher`):

```
// ============================================================
// GVDM11 – Bases de datos orientadas a grafos (Neo4j)
// Script de creación de la base de datos de la red de lectores
// GVDM11 - Álvaro Ramos Bernabé
// ============================================================

// 0) Limpieza (permite relanzar el script)
MATCH (n) DETACH DELETE n;

// 1) Restricciones de unicidad (crean además un índice)
CREATE CONSTRAINT persona_nombre IF NOT EXISTS FOR (p:Persona) REQUIRE p.nombre IS
UNIQUE;
CREATE CONSTRAINT libro_id IF NOT EXISTS FOR (l:Libro) REQUIRE l.id IS UNIQUE;

// 2) Nodos Persona
CREATE (:Persona {nombre: 'Juan',     edad: 22}),
       (:Persona {nombre: 'María',    edad: 27}),
       (:Persona {nombre: 'Luisa',    edad: 32}),
       (:Persona {nombre: 'Alberto',  edad: 29}),
       (:Persona {nombre: 'Fernando', edad: 47});

// 3) Nodos Libro, Autor y Editorial
CREATE (l1:Libro {id: 1, titulo: 'El médico', anio: 2008}),
       (l2:Libro {id: 2, titulo: 'La pirámide inmortal', anio: 2015}),
       (a1:Autor {nombre: 'Noah Gordon'}),
       (a2:Autor {nombre: 'Javier Sierra'}),
```

```
       (e1:Editorial {nombre: 'Rocabolsillo'}),
       (e2:Editorial {nombre: 'Planeta'}),
       (l1)-[:ESCRITO_POR]->(a1),
       (l2)-[:ESCRITO_POR]->(a2),
       (l1)-[:PUBLICADO_POR]->(e1),
       (l2)-[:PUBLICADO_POR]->(e2);

// 4) Amistades (relación simétrica: se guarda una vez y se consulta sin dirección)
UNWIND [['María','Alberto'], ['María','Fernando'],
        ['Luisa','Alberto'], ['Luisa','Juan'],
        ['Juan','Alberto'],  ['Juan','Fernando']] AS par
MATCH (a:Persona {nombre: par[0]}), (b:Persona {nombre: par[1]})
CREATE (a)-[:AMIGO_DE]->(b);

// 5) Lecturas con fecha y valoración como propiedades de la relación
UNWIND [
  {p: 'Juan',     l: 1, fecha: '2015-02-16', val: 9},
  {p: 'María',    l: 1, fecha: '2015-07-15', val: 8},
  {p: 'Fernando', l: 2, fecha: '2015-04-08', val: 7},
  {p: 'Luisa',    l: 2, fecha: '2016-03-16', val: 9},
  {p: 'María',    l: 2, fecha: '2016-09-15', val: 8}
] AS r
MATCH (p:Persona {nombre: r.p}), (l:Libro {id: r.l})
CREATE (p)-[:HA_LEIDO {fecha: date(r.fecha), valoracion: r.val}]->(l);
```

```
aramber@localhost:~$ cypher-shell -a neo4j://localhost:7687 -f crear_bd_lectores.cypher --format verbose
0 rows
ready to start consuming query after 439 ms, results consumed after another 0 ms
0 rows
ready to start consuming query after 164 ms, results consumed after another 0 ms
Added 1 constraint
0 rows
ready to start consuming query after 64 ms, results consumed after another 0 ms
Added 1 constraint
0 rows
ready to start consuming query after 256 ms, results consumed after another 0 ms
Created 5 nodes, set 10 properties, added 5 labels
0 rows
ready to start consuming query after 188 ms, results consumed after another 0 ms
Created 6 nodes, created 4 relationships, set 10 properties, added 6 labels
0 rows
ready to start consuming query after 492 ms, results consumed after another 0 ms
Created 6 relationships
0 rows
ready to start consuming query after 437 ms, results consumed after another 0 ms
Created 5 relationships, set 10 properties
```

Figura 3. Ejecución del script con `cypher-shell -f crear_bd_lectores.cypher`: 11 nodos creados y 15 relaciones.

```
aramber@localhost:~$ cypher-shell --format verbose 'MATCH (n) RETURN labels(n)[0] AS etiqueta, count(*) AS nodos ORDER BY etiqueta;'
+--------------------+
| etiqueta    | nodos |
+--------------------+
| "Autor"     | 2     |
| "Editorial" | 2     |
| "Libro"     | 2     |
| "Persona"   | 5     |
+--------------------+

4 rows
ready to start consuming query after 142 ms, results consumed after another 10 ms
aramber@localhost:~$ cypher-shell --format verbose 'MATCH ()-[r]->() RETURN type(r) AS relacion, count(*) AS total ORDER BY relacion;'
+------------------------+
| relacion        | total |
+------------------------+
| "AMIGO_DE"      | 6     |
| "ESCRITO_POR"   | 2     |
| "HA_LEIDO"      | 5     |
| "PUBLICADO_POR" | 2     |
+------------------------+

4 rows
ready to start consuming query after 79 ms, results consumed after another 3 ms
```

Figura 4. Comprobación: nodos por etiqueta y relaciones por tipo.

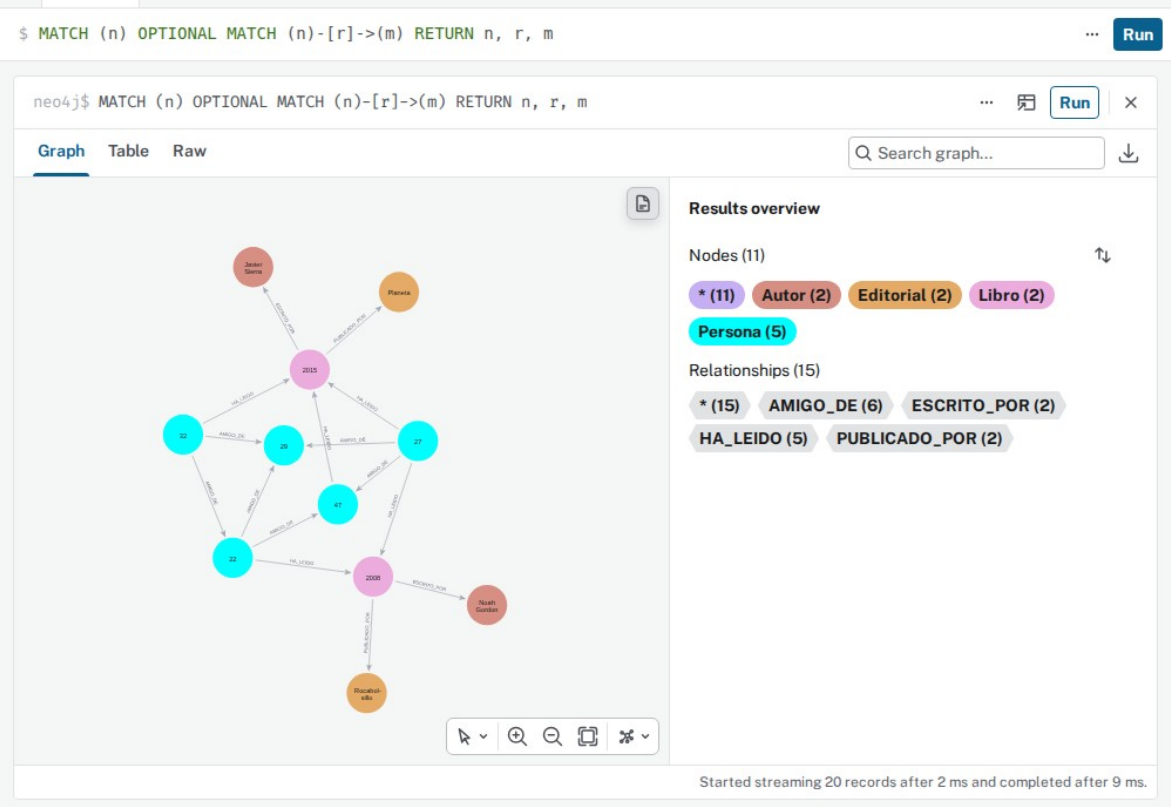


Figura 5. El grafo completo en Neo4j Browser (etiquetas y relaciones en el panel *Results overview*).

2) Consultas en Cypher

```
// Consulta 1: ¿Quiénes leyeron el libro #2?
MATCH (p:Persona)-[r:HA_LEIDO]->(l:Libro {id: 2})
RETURN p.nombre AS lector, r.fecha AS fecha, r.valoracion AS valoracion
ORDER BY r.fecha;

// Consulta 2: ¿Cuál es la valoración promedio del libro #1?
MATCH (:Persona)-[r:HA_LEIDO]->(l:Libro {id: 1})
RETURN l.titulo AS libro, avg(r.valoracion) AS valoracion_media, count(r) AS
num_valoraciones;

// Consulta 3: ¿Quién es el usuario de mayor edad?
MATCH (p:Persona)
RETURN p.nombre AS nombre, p.edad AS edad
ORDER BY p.edad DESC
LIMIT 1;
```

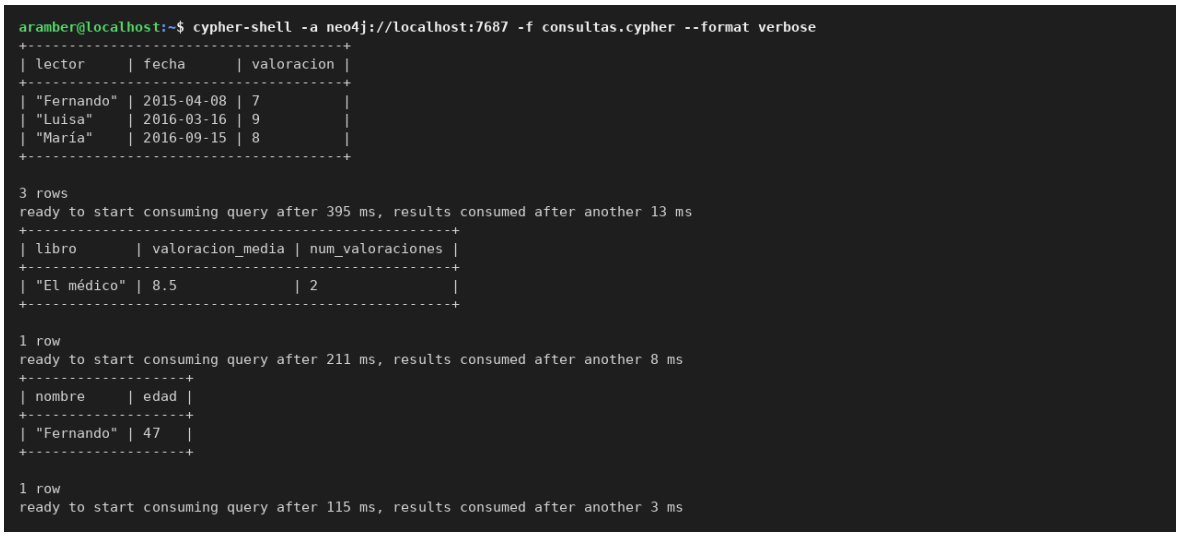


Figura 6. Resultado de las tres consultas.

- **¿Quiénes leyeron el libro #2?** Fernando (08/04/2015, nota 7), Luisa (16/03/2016, nota 9) y María (15/09/2016, nota 8).
- **Valoración promedio del libro #1** (*El médico*): **8,5**, media de las notas de Juan (9) y María (8), calculada con `avg()`.
- **Usuario de mayor edad: Fernando, 47 años**. Se ordena por edad descendente con `ORDER BY` y se toma el primero con `LIMIT 1`.

Referencias

[1] Neo4j Deployment Center – Community Edition. https://neo4j.com/deployment-center/

[2] Neo4j Operations Manual – Installation (Linux tarball) y requisitos de Java. https://neo4j.com/docs/operations-manual/current/installation/

[3] Documentación de Azure Cosmos DB for Apache Gremlin; ArangoDB Documentation; OrientDB Manual.